漫步数码时代，

铸就精彩人生，

程序员修炼之路，

从精雕算法开始……

第4章 贪心法

（Greedy Approach）

（一）

主讲人：周正深

4.1 贪心法的设计思想

4.2 贪心法的正确性证明

4.3 对贪心法得不到最优解情况的处理

4.4 贪心法的典型应用

4.4.1 最优前缀码

4.4.2 最小生成树

4.4.3 单源最短路径

例4.1 活动选择问题

输入：$S=\{1, 2, \ldots, n\}$为n 项活动的集合，s_i, f_i 分别为活动 i 的开始和结束时间，活动 i 与 j 相容 $\Leftrightarrow$ $s_i \geq f_j$ 或 $s_j \geq f_i$.

求：最大的两两相容的活动集 A

实例

i	1	2	3	4	5	6	7	8	9	10
s_i	1	3	2	5	4	5	6	8	8	2
f_i	4	5	6	7	9	9	10	11	12	13

普通策略1：按开始时间排序使得 $s_1 \leq s_2 \leq \ldots \leq s_n$，从前向后挑选

普通策略2：按占用时间排序使得 $f_1 - s_1 \leq f_2 - s_2 \leq \ldots \leq f_n - s_n$，从前向后挑选

贪心策略：按结束时间排序使得 $f_1 \leq f_2 \leq \ldots \leq f_n$，从前向后挑选。

以上策略中的挑选都要注意满足相容性条件

普通策略1：按开始时间排序使得 $s_1 \le s_2 \le \ldots \le s_n$，从前向后挑选

i	1	3	10	2	5	4	6	7	8	9
s_i	1	2	2	3	4	5	5	6	8	8
f_i	4	6	13	5	9	7	9	10	11	12

1 → 5　解 $A = \{1, 5\}$

普通策略2：按占用时间排序使得 $f_1 - s_1 \le f_2 - s_2 \le \ldots \le f_n - s_n$，从前向后挑选

i	2	4	1	8	3	6	7	9	5	10
s_i	3	5	1	8	2	5	6	8	4	2
f_i	5	7	4	11	6	9	10	12	9	13
t_i	2	2	3	3	4	4	4	4	5	11

2 → 4 → 8　解 $A = \{2, 4, 8\}$

贪心策略：按结束时间排序使得 $f_1 \le f_2 \le \ldots \le f_n$，从前向后挑选

以上策略中的挑选都要注意满足相容性条件

i	1	2	3	4	6	5	7	8	9	10
s_i	1	3	2	5	5	4	6	8	8	2
⇨ f_i	4	5	6	7	9	9	10	11	12	13

1 → 4 → 8 解 $A = \{1, 4, 8\}$

贪心策略可获得最优解！

下面通过反例来证明策略1和策略2均不能获得最优解。

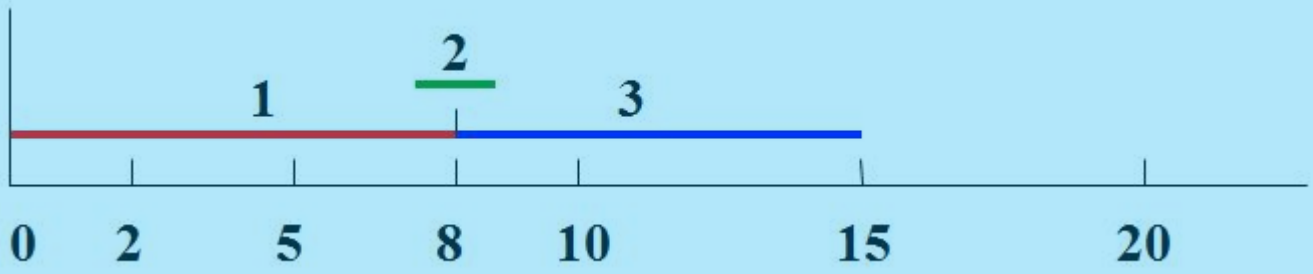
例4.1：活动选择问题-两个反例

策略1反例：$S=\{1, 2, 3\}$，$s_1=0$, $f_1=20$, $s_2=2$, $f_2=5$, $s_3=8$, $f_3=15$

开始最早优先

策略2反例：$S=\{1, 2, 3\}$，$s_1=0$, $f_1=8$, $s_2=7$, $f_2=9$, $s_3=8$, $f_3=15$

用时最短优先



例4.1：贪心算法

算法4.1 GreedySelect

输入：活动集 S，s_i，f_i，$i=1, 2, \ldots, n$，且 $f_1 \le \ldots \le f_n$ (已排序)

输出：$A \subseteq S$，选中的活动子集

```
n ← length[S]    // n 活动个数，s_i 开始时间，f_i 结束时间
A ← {1}
j ← 1            //已选入的当前最后一个活动的标号
for i ← 2 to n do
    if s_i ≥ f_j then //判断相容性
        A ← A ∪ { i }
        j ← i
return A
```

最后完成时间 $t = \max\{f_k : k \in A\}$

例4.1：算法运行实例

输入：$S = \{1, 2, \ldots, 10\}$

i	1	2	3	4	5	6	7	8	9	10
s_i	1	3	0	5	3	5	6	8	8	2
f_i	4	5	6	7	8	9	10	11	12	13

解：$A = \{1, 4, 8\}$，$t = 11$

时间复杂度：排序+活动选择=$O(n\log n)+O(n)=O(n\log n)$

问题：如何证明该算法对所有的实例都能得到正确的解？

定理4.1 算法Select(贪心法) 执行到第 k 步, 选择 k 项活动 $i_1 = 1$, i_2 , … , i_k, 那么存在最优解 A 包含 $i_1=1, i_2, \ldots, i_k$ 。

(注：其中i_1固定为1)

根据定理，算法至多到第 n 步得到最优解：

$S=\{1, i_2, \ldots, i_k\}$是活动集，且 $f_1 \le \ldots \le f_k$

定理的证明过程：

需要先证明下列引理1成立。

引理1. 活动安排问题最优解中包含活动1(按结束时间增序排列)。

1. 引理：活动安排问题最优解中包含活动1(按结束时间增序排列)。

引理的证明（不用数学归纳法，直接证明）：

①k=1时, 最优解显然包含活动 1.

②k>1时,任取最优解A, A中的活动按截止时间递增排列. 如果A的第一个活动为j，$j \neq 1$, 令

$$A' = (A - \{j\}) \cup \{1\},$$

$|A|$表示A中所含活动数

∵ $f_1 \leq f_j$, 所以A'也是相容活动集合

∵ $|A'|=|A|$, A'也是最优解，且含有1.

引理证毕。

例4.1：算法正确性证明(续)

2. $S=\{1, i_2, \ldots, i_k\}$，且 $f_1 \le \ldots \le f_k$ 活动集是最优解

用数学归纳法证明：

①k=1，命题显然成立

②假设命题对 k 为真, 证明对 k+1 也为真.

假设：算法执行到第 k 步,按结束时间顺序选出来选择了活动 1, $i_2, \ldots, i_k$, 是问题的最优解，即 A 包含 $\{1, i_2, \ldots, i_k\}$。

A中剩下的活动选自集合 $S'=\{ i \mid i \in S, s_i \ge f_k\}$, 且

$A = \{ 1, i_2, \ldots, i_k\} \cup B$

其中B是S'的最优解。

所有起始时间大于第k个活动的结束时间，即剩下与前边选择不冲突的

例4.1：算法正确性证明(续)

2. $S=\{1, i_2, \ldots, i_k\}$，且 $f_1 \le \ldots \le f_k$ 活动集是最优解

根据引理，S'的最优解B含有S'中的第一个活动，即i_{k+1}, 于是

$$\{i_1, i_2, \ldots, i_k\} \cup B = \{i_1, i_2, \ldots, i_k\} \cup \{i_{k+1} \cup (B - \{i_{k+1}\})\}$$
$$= \{i_1, i_2, \ldots, i_k, i_{k+1}\} \cup (B - \{i_{k+1}\})$$

即$\{1, i_2, \ldots, i_k, i_{k+1}\}$是原问题的最优解。

由于i_{k+1}是按结束时间的第k+1次选择，所以前k+1次的选择也是最优解，命题成立。

定理证毕。

上一问题分析

1.贪心法设计思想：

它采用逐步构造最优解的思想，在问题求解的每一个阶段，都作出一个在一定标准下看上去最优的决策；决策一旦作出，就不可再更改；制定决策的依据具有贪心(婪)选择性质。

2.贪心法正确性证明：

(1)数学归纳法、 (2)交换论证法

(1)数学归纳法

①叙述一个描述算法正确性的命题$P(n)$，n为算法步数或者问题规模

②归纳基础：$P(1)$ 或 $P(n_0)$为真，n_0为某个自然数

③归纳步骤：$P(k) \Rightarrow P(k+1)$　　第一数学归纳法

$\forall k(k<n)P(k) \Rightarrow P(n)$　　第二数学归纳法

归纳法1：$P(1) \Rightarrow P(2) \Rightarrow P(3) \Rightarrow P(4) \cdots$

归纳法2：

$$\begin{array}{ccccc} & & & P(1) & \\ & & P(1) & P(2) & \\ P(1) & \Rightarrow & P(2) & \Rightarrow P(3) & \Rightarrow P(4) \cdots \end{array}$$

(2)交换论证

①分析算法的解的结构特征

②从一个最优解逐步进行结构变换(替换成分、交换次序等)得到一个新的解（结构上与贪心算法的解更接近）

③证明：上述变换最终得到算法的解，且变换在有限步结束，每步变换都保持解的最优性不降低.

参见例4.1 引理1的证明

参见例4.3 贪心法的解的性质：没有空闲时间，没有逆序。

逆序(i, j)：$d_i < d_j$，且$f_i > f_j$， d为活动安排的开始时间，f为活动本身的结束时间。(s_i为每个活动要求的开始时间)

4.1 贪心法的设计思想

3. 贪心法的基本要素：

(1) 贪心选择性质(关键因素：必须满足贪心选择性质)

贪心选择性质是指所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来达到。

(2) 最优子结构性质

当一个问题的最优解包含着它的子问题的最优解时，称此问题具有最优子结构性质。

2.贪心法与动态规划的差异：

贪心算法和动态规划算法都要求问题具有最优子结构性质，这是两类算法的一个共同点。一般能用贪心法求解的问题都可以用动态规划算法求解，但能用动态规划求解的问题不一定能用贪心法进行求解。

4.1 贪心法的设计思想-贪心算法的特点

4.贪心法（又称贪婪法）的特点：

①**逐步构造最优解** 每一步都是在一定标准下的最优决策，是只顾眼前的最优；

②**不回溯** 贪婪法常以当前情况为基础作最优选择，而不考虑整体情况，是一种“短视的”行为，所以贪婪法不要回溯；

③**不可更改** 每一步做出的决策，在以后的步骤中都不可更改；

④**算法简单** 时间和空间复杂性低。

⑤**需要证明** 贪心法得到最优解必须进行正确性证明。

⑥**满意解** 未加证明的贪婪法是一种不追求最优解，只希望得到较为满意解的方法。贪婪法一般可以快速得到满意的解，省去了为找最优解要穷尽所有可能而必须耗费的大量时间；

贪心法的优势：算法简单，时间和空间复杂性低。

5. 贪心法的解题步骤

(1) 数学模型 建立数学模型来描述问题；

(2) 分解子问题 把求解的问题分成若干个子问题；

(3) 求解子问题 对每一子问题求解，得到子问题的局部最优解；

(4) 合成原问题 把子问题的解（局部最优解）合成原来解问题的一个解；

(5) 算法正确性证明。

例4.2：最优装载 Loading

例4.2 最优装载

n 个集装箱1, 2, …, n 装上轮船，集装箱 i 的重量 w_i, 轮船装载重量限制为C，无体积限制。问如何装载使得上船的集装箱数量最多？不妨设 $w_i \le C$，$\Sigma w_i > C$。

这个问题是0-1背包问题的子问题(特例)：

集装箱相当于物品，物品重量是w_i，价值v_i=1（相当于价值都相同），轮船载重C相当于背包重量限制b。

例4.2：最优装载 Loading

1.建模：

目标函数 $\max \sum_{i=1}^{n} x_i \qquad x_i = 0,1 \quad i = 1,2,\ldots, n$

约束条件 $\sum_{i=1}^{n} w_i x_i \le C$

2.算法设计：

贪心策略：将集装箱按照从轻到重排序，轻者先装。

3.算法复杂度：等于排序复杂度$O(n\log n)$

最优装载正确性证明

定理4.2 对装载问题任何规模为n的输入实例，算法得到最优解.

证明 对问题规模归纳. 设集装箱从轻到重记为1, 2, … ,n.

归纳基础： n=1, 只有1个箱子，算法显然正确.

归纳步骤：假设对于任何n个集装箱的输入实例，贪心法都能得到最优解，那么对于任何n+1个箱子的输入实例，贪心法也能得到最优解。

证：$N = \{1, 2, \ldots, n+1\}$, 其中 $w_1 \le w_2 \le \ldots \le w_{n+1}$，载重限制$C$

去掉箱子1，原问题转化为规模n的子问题（N'，C'）

设：$N' = \{2,3,\ldots,n+1\}$，$C' = C - w_1$, 贪心算法得到最优解I'

即： I' 是 N' 和 C' 的最优解----------①

最优装载正确性证明

回到原来规模N (即n+1输入的实例)：

令 $I = \{1\} \cup I'$，则 I (这正是贪心法的解)是关于 N 的最优解。

反证：假若存在N 的最优解 I^*，I^*中含1，且$|I^*| > |I|$。

(如果 I^* 中没有1，用 1 替换 I^* 中的第一个元素得到的解也是最优解，即前边的引理)；

显然可知 $I^* - \{1\}$是关于子问题（N'，C'）的解，此解集装箱的个数 $|I^* - \{1\}| > |I - \{1\}| = |I'|$

$\therefore$ I' 不是 N' 和 C' 的最优解----------②

②式与 ①式矛盾

问题得证，完毕。

第4章 习题4 P123 4.1（上机编程）

4.1 设有n个顾客同时等待一项服务，顾客i需要的服务时间为t_i，$i=1,2,\cdots,n$。从时刻0开始计时,若在时刻t开始对顾客i服务,那么i的等待时间就是t。应该怎样安排n个顾客的服务次序使得总的等待时间(每个顾客等待时间的总和)最少。

(1)使用贪心法求解这个问题时的贪心选择策略是什么？

(2)写出贪心法的算法描述。

(3)假设服务时间分别为{1,3,2,15,10,6,12}，用贪心法给出这个问题的解。

习题4 P123 4.2

4.2 有n个底面为长方形的物品需要租用库房存放。如果每个物品都必须放在地面上，且所有物品的底面宽度都等于库房的宽度，那么第i个物品占据库房面积大小只需要用它的底面长度l_i来表示，i=1, 2，…n，设库房总长度是L，且$\sum_{i=1}^{n} l_i > L$。如果要求放入库房的物品个数最多，选用哪种算法设计技术？简述算法的设计思想，证明算法的正确性，并估算算法最坏情况下的时间复杂性。

结 束

谢谢收看，再见！